



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Practical No. 8

AIM: A college admission portal requires students to enter valid information before submitting their registration forms. Build a Student Registration Form with JavaScript validation to verify mandatory fields, email format, password length, and mobile number before submission.

NAME OF STUDENT	
USN NO.	
DATE OF PERFORMANCE	
DATE OF SUBMISSION	

AIM: A college admission portal requires students to enter valid information before submitting their registration forms. Build a Student Registration Form with JavaScript validation to verify mandatory fields, email format, password length, and mobile number before submission.

OBJECTIVES

- To implement client-side form validation using JavaScript to ensure data integrity and improve user experience before server transmission.
- To apply Regular Expressions (RegEx) for matching and validating complex input patterns such as email syntax and mobile number constraints.
- To intercept default form submission behavior using `event.preventDefault()` and dynamically display contextual error feedback on the web interface.

THEORY

Client-side validation serves as the first line of defense in modern web applications, providing instant feedback to users while preventing malformed or incomplete data from reaching the server. Building a robust validation system for a student registration portal relies on four foundational concepts of JavaScript, the Document Object Model (DOM), and pattern matching:

- **DOM Access and Input Value Extraction:** Form inputs store user-provided data within their `.value` property. JavaScript targets these fields using methods like `document.getElementById()` or `document.querySelector()`. Before performing validation checks, it is standard practice to use the `.trim()` method to strip leading and trailing whitespace, ensuring that fields containing only spaces are correctly identified as empty.
- **Conditional Logic and Mandatory Field Verification:** Conditional statements (`if-else`) evaluate whether essential fields contain valid data. Mandatory field checks ensure that text boxes are not left blank and that selection dropdowns or checkboxes meet required criteria. If a field fails a check, a corresponding error flag is raised, and submission is blocked.
- **Regular Expressions (RegEx) for Pattern Matching:** Complex data formats cannot be verified through simple length or equality checks. Regular expressions provide a pattern-matching syntax used to validate specific structures. For example, an email validation RegEx checks for local-name characters, an "@" symbol, a domain name, and a top-level domain (e.g., `^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$`). Similarly, mobile number validation checks for exact digit counts (such as a 10-digit numeric sequence).
- **Event Interception and Dynamic Feedback Rendering:** When a user clicks the submit button, the form triggers a `submit` event. By attaching an event listener and

invoking `event.preventDefault()`, the script halts the default page reload behavior. If validation errors exist, error messages are dynamically injected into designated feedback container elements (using `textContent` or class modifications), alerting the user precisely which fields require correction.

PRACTICAL STEPS

1. **Create the HTML Structure:** Setup the registration form with input fields for full name, email address, password, and mobile number, alongside a submit button and error span tags for each field.
2. **Style the Interface with CSS:** Apply styling rules to highlight valid inputs (e.g., green borders) and invalid inputs (e.g., red borders with visible error text underneath).
3. **Bind the Event Listener in JavaScript:** Select the form element and attach an event listener targeting the submit event.
4. **Intercept Default Submission:** Call `event.preventDefault()` inside the event handler function to prevent automatic form submission before validation logic completes.
5. **Execute Validation Logic:** Extract input values, use `trim()`, evaluate length constraints for passwords, and test email and mobile formats using Regular Expressions.
6. **Render Dynamic Feedback and Submit:** Display error messages next to invalid fields if checks fail; if all inputs pass validation, allow the form submission to proceed or trigger a success state.

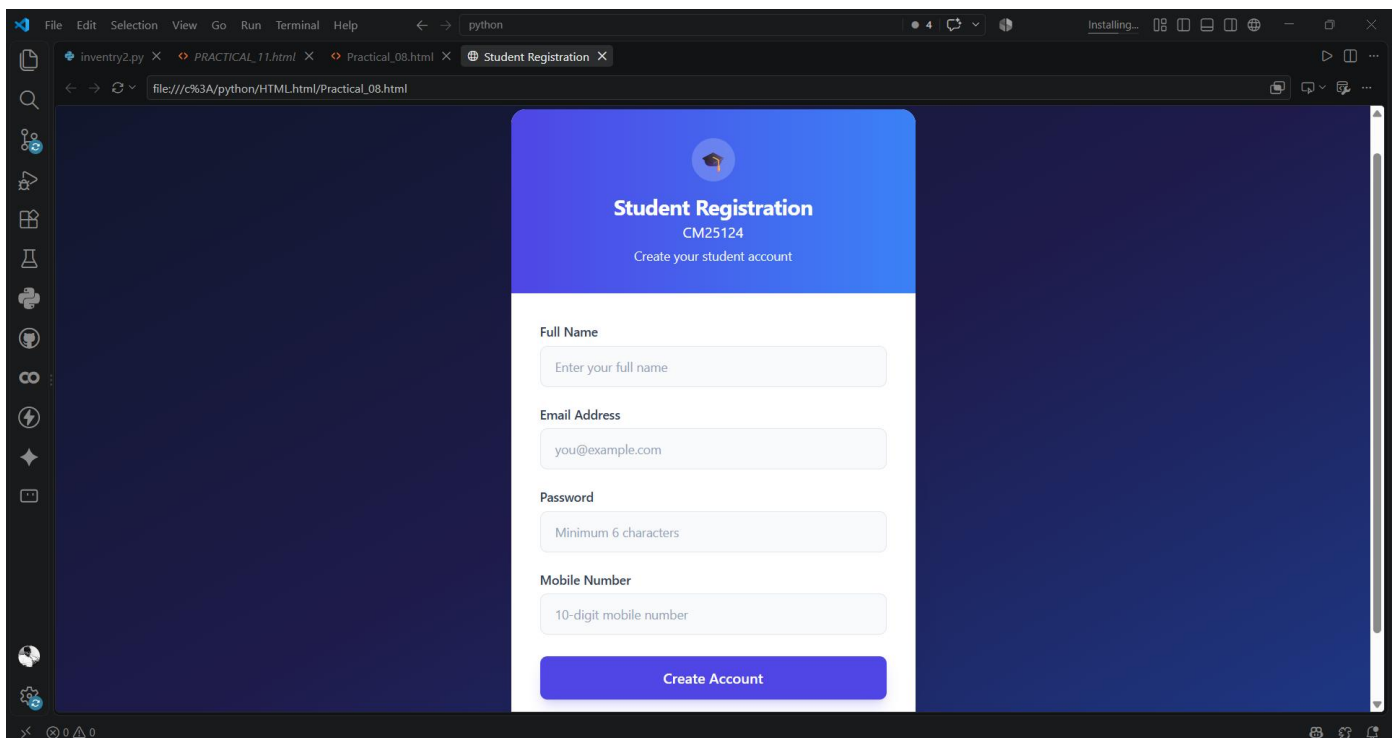
SOURCE CODE GITHUB REPOSITORY

<https://github.com/Vishesh786900/Modern-Web-dev>

SOURCE CODE GITHUB REPOSITORY QR CODE



OUTPUT SCREENSHOTS



REVIEW QUESTIONS

1. Why is client-side validation considered a supplementary user-experience layer rather than a replacement for server-side validation?

2. How does the `RegExp.test()` method function when validating structured inputs like email addresses and phone numbers?

3. What is the security risk of relying solely on client-side JavaScript validation for user registration systems?

4. Why is using `.trim()` critical before checking the length or emptiness of a text input field?

5. How does `event.preventDefault()` allow JavaScript to control whether a form successfully transmits data to a server?

CONCLUSION

REFERENCES

- Mozilla Developer Network (MDN) Web Docs – Client-side form validation:
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Forms/Form_validation
- Mozilla Developer Network (MDN) Web Docs – Regular Expressions:
https://developer.mozilla.org/enUS/docs/Web/JavaScript/Guide/Regular_expressions
- W3Schools – JavaScript Validation API:
https://www.w3schools.com/js/js_validation_api.asp

EVALUATION

Regular Performance in Lab (3)	
Viva-Voce (3)	
Quality of Submission (4)	
Faculty Signature & Date	